Programming Lab – Argumentation Technology

Introduction

Carlotta Quensel

c.quensel@ai.uni-hannover.de

April 15, 2026

Natural Language Processing

Organization

General Information

Topic

- Computational analysis and synthesis of natural language arguments
- Python programming assignments of state-of-the-art NLP methods
  ➜ new NLP method and argumentation task in each assignment

Recommended Background

- Programming knowledge in Python and basic NLP/ML terms and methods
  (assignment-specific techniques are introduced but basic DL/ML knowledge is expected)
  ➜ Lab: Human Language Technology/any course with NLP programming
- Lecture: Computational Argumentation – parallel to this course; broader and more in-depth theoretical discussion of computational argumentation

Literature

- Sarkar. 2019. *Text Analytics with Python*.
- Stede and Schneider. 2019. *Argumentation Mining*.
- Tunstall et al. 2022. *Natural Language Processing with Transformers*.

Semester Schedule

1. Warm-up	○	15/04	Introduction & organization	1101.F138
	○	22/04	Presentation (submission: 20/04, 23:59h)	1101.F138
2. Data ✔	●	29/04	Introduction	1101.F138
	○	06/05	Q&A	1101.F138
	●	13/05	Presentation (submission: 11/05, 23:59h)	**!**3408.010
3. Mining ✔	●	20/05	Introduction	**!**3408.010
		~~27/05~~	*Pentecost week* (Q&A via StudIP)	——
	●	03/06	Presentation (submission: 01/06, 23:59h)	1101.F138
4. Assessment ✔	●	10/06	Introduction	1101.F138
	○	17/06	Q&A	1101.F138
	●	24/06	Presentation (submission: 22/06, 23:59h)	1101.F138
5. Generation ✔	●	01/07	Introduction	1101.F138
	○	08/07	Q&A	1101.F138
	●	15/07	Presentation (submission: 13/07, 23:59h)	1101.F138

✔: graded assignment; ●: mandatory attendance; **!**: sessions start at 15:30

Lab Structure

- Structured into Python assignments on JupyterHub that determine if you pass the course.
- First assignment is ungraded to familiarize yourself with the format
- Graded assignments each address different CA tasks in a three-week cycle:

Sessions

Intro to the topic and technique, explanations of each task
Q&A for you to ask questions that arose while working on the tasks
Present your own submitted solution (3-6 students per assignment)

Submissions

- Assignments are released, submitted, and graded via JupyterHub
- Submit in pairs, only once per group, and on time
  - no solo submissions – try the Stud-IP forum, then email me
  - no double submissions – we cannot decide which solution should count
  - JupyterHub only saves the latest version, do not resubmit after the deadline
- The submitted Jupyter Notebook should include:
  - reproducible and working code with explanatory comments
  - proper documentation/theory tasks in full sentences and markdown format
  - declaration on AI use (task-specific: if, how, and why was AI used)

Grading

- Each assignment is worth 30 points ➜ 120 points overall
  - Requirements for points will be stated in each task
  - To pass the course, you will need more than 80 points overall
- Each student must present a (sub)task
  - Explain and motivate your submitted solution
  - You must present alone, and you can repeat a failed presentation once (2 failed presentations → failed course)
- The lab is ungraded

Policy on using AI

We do not forbid the use of AI tools for your tasks

- We require you to be authors of your solutions (texts, code, plots. . .).
- Any use for your solutions must be reasoned as part of a submission.
- Clear authorship violations (according to our judgment) lead to failing.

We do *not* generally encourage the use of AI tools

- Intended learning effects may not happen when you use AI tools.
- Other risks include scientific misconduct, plagiarism, loss of skills, . . .
- Recently, unconsiderate use has led to notably higher fail rates!

We encourage learning a considerate use of AI tools

✔ Use that supports your own solutions:

> Here is my solution of the task of <TASK>: <SOLUTION>. Am I missing something?

✘ Use that take over your tasks:

> Please solve the following task for me: <TASK>. Adjust the solution to this <SOURCE>.

! Always double check tool output against other sources, common sense, . . .

Policy on using AI

Examples of using AI to support tasks

- ✔ Optimizing linguistic *details* of your own texts
- ✔ Creating format templates, e.g., LaTeX/markdown tables
- ✔ Creating recurring boilerplate code (auxiliary only)
- ✔ Getting explanations of unclear concepts
- ✔ Getting feedback to your own solutions
- ✔ Using the tools' explicit learning support modes

Examples of using AI to take over tasks

- ✘ Directly creating solutions to your tasks
- ✘ Creating new content that you use in your texts
- ✘ Carry out numeric computations
- ✘ Several other uses; in doubt, ask us!

Learn more: LUH page on legal and technical aspects of AI use;
U Vienna page with useful hints on AI use

Organization
General Info
Schedule
Submissions
Grading
AI Policy
Tutorials
JupyterHub
Intro to NLP
A1 Warmup
Overview
Tasks
Mock Grading
Additional Info

Tutorials

Let's move to JupyterHub!

JupyterHub

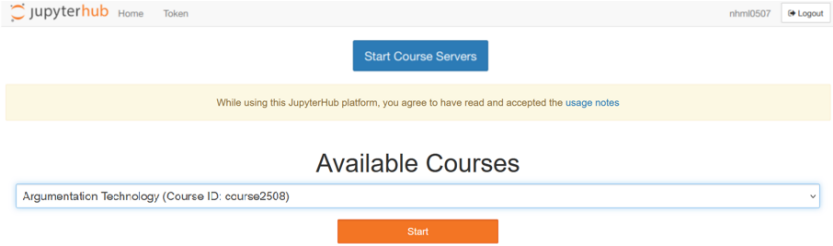
`https://jupyterhub.cluster.uni-hannover.de`

- Only accessible from eduroam, use a VPN to access it from other locations.
- Notebooks have a 10s autosave interval, but you should save manually to avoid progress loss due to connection or kernel issues.
- JupyterHub has no internet access and `pip` downloads are not persistent. Installation requests should be posted to the Stud-IP forum.

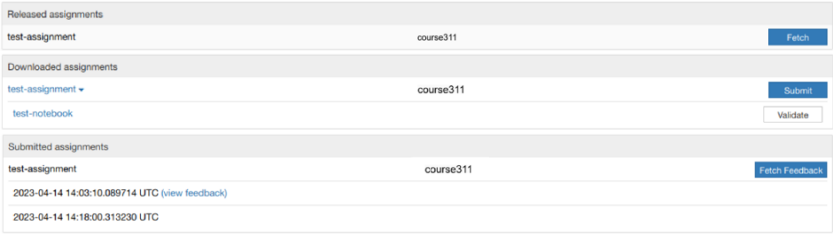
If you haven't received credentials to the JupyterHub, let us know immediately.

JupyterHub

- Access the course



- Everything important happens in the Assignments tab:



Intro to NLP

- The methods shown in the tutorial are expected basic knowledge
- You can find the tutorial on JupyterHub and on Stud-IP
- The notebook is accessed like an assignment, but never collected/graded

Organization
General Info
Schedule
Submissions
Grading
AI Policy
Tutorials
JupyterHub
Intro to NLP
A1 Warmup
Overview
Tasks
Mock Grading
Additional Info

Assignment 1 – Warmup

Overview

Topics Covered

- List comprehensions
- String manipulation
- Data reading and preprocessing
- Computing and plotting basic statistics
- Training and evaluating machine learning models

Learning Goals

- Practice Python and NLP Basics covered in the introductory tutorials
- Get to know JupyterHub and the submission process
- Practice presenting solutions (next session)

Deadline: Tuesday, April 21, 2025 at 23:59h

Task 1 – List Comprehension

a. Using list comprehension, create a `newlist` from the given `numbers`, where each even element is squared and each odd element is cubed.

```
numbers = [4, -23, 7, 68, -12, 1, 12]
# YOUR CODE HERE
```

(2 points)

b. Rewrite the following loop using list comprehension for a `new_number_list`.

```
number_list = []
for x in range(100):
    if x % 3 == 0:
        if x % 5 == 0:
            number_list.append(x)
# YOUR CODE HERE
```

(2 points)

Task 2 – String Manipulation

Write a function that accepts a sentence, calculates the number of upper case and lower case letters, and returns both as a tuple.

```python
sent = "One day I am going to make the ONIONS cry"

def count_cased_letters(sentence: str) -> tuple:
    """Count the upper and lower case letters of the input string.
    :param sentence: string for which to count letters
    :return: tuple of the number of upper and lower case letters
    """
    # YOUR CODE HERE
```

(2 points)

Task 3 – Maths with Numpy

Write a function that computes the dot product of two matrices by looping over both indexes and multiplying the individual entries. Compare your results with the built-in numpy function `np.dot(m1, m2)`

```
import numpy as np

m1 = np.array([[1,2,3],[4,5,6]])
m2 = np.array([[7,8],[9,10],[11,12]])
m3 = np.array([[7,8,9]])

def dot_product(x: np.ndarray, y: np.ndarray) -> np.ndarray:
    """Calculate the dot product of two input arrays.
    :param m1, m2: two numpy arrays of integers
    :return: a numpy array of integers
    """
    # YOUR CODE HERE
```

(4 points)

Task 4 – Data Statistics and Machine Learning

In the directory `./data/`, you will find the dataset IBM Argumentative Sentences in Spoken Language (Shnarch et al., 2020) in a `csv` file as well as a Readme file describing the data. After reading the data into a `pandas` DataFrame, complete the following tasks:

a. Calculate the number of words, average word length, and number of unique words for each sentence, for the whole dataset, and for each topic in the dataset.

b. Plot the above distributions for the debate topics using `matpotlib` or `seaborn`.

(10 points)

Task 4 – Data Statistics and Machine Learning

c. Train a multiclass classifier that predicts the topic of a provided sentence using `scikit-learn`.
   - Use only topics with more than 20 samples
   - Do not forget to split the data into train/test splits (70%/30%). Stratify the splits based on the topic to ensure that the debate topic distribution is the same in both splits.
   - You can use any classification approach available from the package, a helpful guide is available in the package tutorials.
   - After training, report precision, recall, and F_1 for each topic and plot a confusion matrix.

(10 points)

Grade Requirements

Note! Due to the basic programming tasks and the shorter time frame, the points are 'cheaper' than those of future assignments.

- Points given for running code without preventable errors, following the task description in using/not using packages/methods as indicated
- Points given for correct outputs and correct full-text answers
- Points deducted for incorrect outputs or calculations
- Points deducted for sloppy outputs, e.g., individually printed numbers without context, confusion matrix without any labels, etc.
- Clear authorship violations (plagiarism, AI solutions) fail without grading

Additional Information

- You can reuse and build upon the data processing and modeling solutions from the tutorials.
- Make sure to justify and explain your implementation choices properly in the Jupyter Notebook either as comments to your code or, if applicable, in separate markdown cells.
- If you need external libraries and packages that are not available on our JupyterHub instance, reach out on the StudIP forum *until Friday*.

Bibliography

Dipanjan Sarkar. 2019. *Text analytics with Python: A Practitioner's Guide to Natural Language Processing*, 2nd edition. Apress.

Eyal Shnarch, Leshem Choshen, Guy Moshkowich, Ranit Aharonov, and Noam Slonim. 2020. Unsupervised expressive rules provide explainability and assist human experts grasping new domains. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2678–2697, Online. Association for Computational Linguistics.

Manfred Stede and Jodi Schneider. 2019. *Argumentation Mining*, volume 40 of *Synthesis Lectures on Human Language Technologies*. Morgan & Claypool.

Lewis Tunstall, Leandro von Werra, and Thomas Wolf. 2022. *Natural Language Processing with Transformers: Building Language Applications with Hugging Face*, revised edition. O'Reilly Media.